# Database Development and Design (SOE201)

## Lecture 6a: Introduction to Query Optimisation 1

Dr. Wei Wang

AIAT

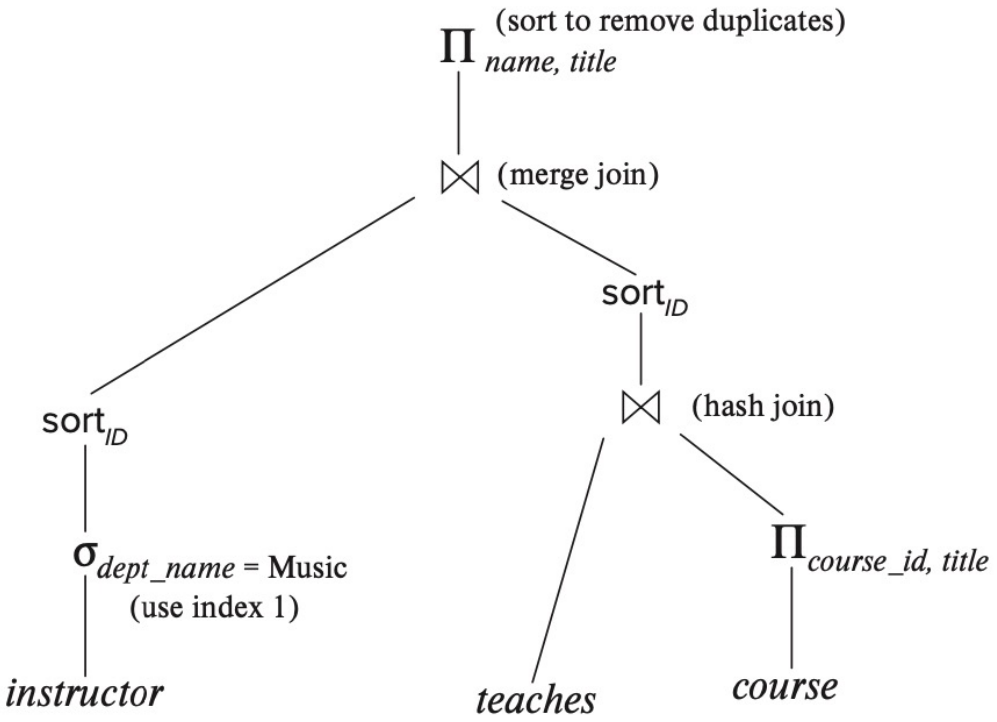
# Learning Outcomes

- Introduction to Query Optimisation
  - Materialisation and Pipelining
  - Equivalence Rules
  - Transformation of Relational Expressions
  - Generation of Equivalent Expressions

Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Evaluation of Expressions

- So far we have seen algorithms for individual operators
  - These have then to be *combined to evaluate complex expressions with multiple operations*.




Xi'an Jiaotong-Liverpool University 西交利物浦大学

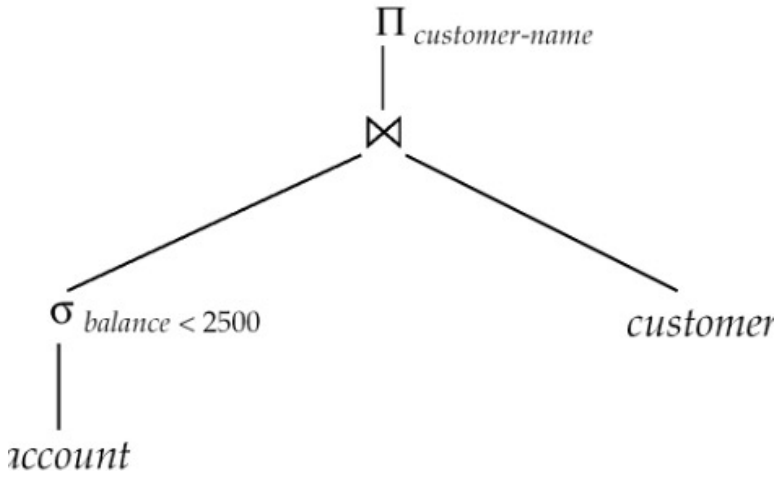
# Evaluation of Expressions – cont'd

- Alternatives for evaluating expression tree
  - **Materialisation**: generate results for expression whose inputs are relations or relations that are already computed. Temporary relations must be materialised (stored) on disk.
  - **Pipelining**: pass on tuples to parent operators even as operation is being executed (no need to store temporary results onto disk).

Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Materialisation

- Materialised evaluation: Use intermediate results materialised into temporary relations to evaluate next-level operations.
  - e.g. compute and store selection (*treat it as new relation*), then compute join with *customer* and store result, and finally compute projections on *customer-name.*




Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Materialisation cont'd

- Materialised evaluation is always applicable.
- May require considerable storage space: cost of writing results to disk and reading them back can be quite high
- Cost formulas learned so far ignore cost of writing final results to disk, so
  - Overall cost = Sum of costs of individual operators + cost of writing intermediate results to disk

Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Pipelining

- Pipelined evaluation: evaluate several operations simultaneously, passing results of one operation on to next.
  - e.g. in previous expression tree, don't store result of selection
    - instead, pass tuples directly to join.
    - Similarly, don't store result of join, pass tuples directly to projection.
- Cheaper than materialisation: no need to store temporary relation to disk.
- However, pipelining may not always be possible, e.g. external merge-sort and hash-join where preliminary phase is required over whole relations.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

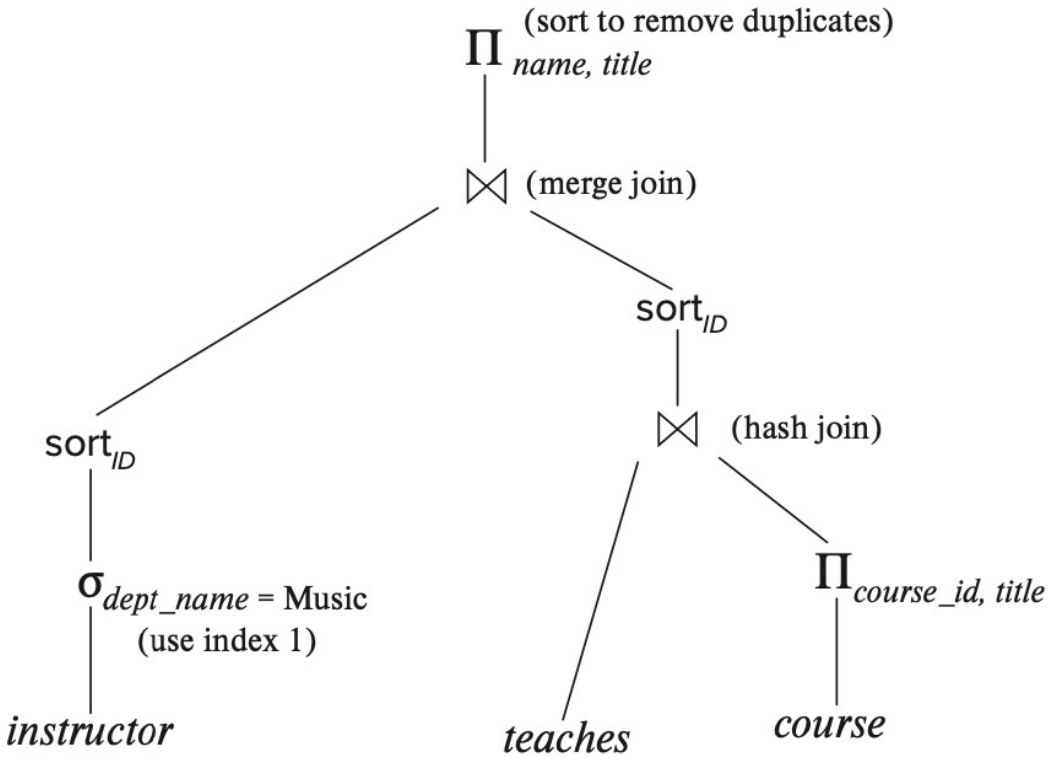
# Pipelining – cont'd

- Pipelines can be executed in two ways: demand driven and producer driven
- Producer-driven (or eager or push)
  - Operators produce tuples eagerly and pass to parents
    - Buffer maintained between operators, child puts tuples in buffer, parent removes tuples from buffer
    - If buffer is full, child waits till there is space in buffer, and then generates more tuples
- Demand driven (or lazy, or pull)
  - System repeatedly requests next tuple from top level operator
  - Each operation requests next tuple from child operations as required, in order to output its next tuple
  - In between calls, operation has to maintain "state" so it knows what to return next.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Evaluation Plan

- Evaluation plan defines exactly what algorithm is used for each operation, and how execution of operations is coordinated.




Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Notes on Using Pipelining

- If edge in evaluation tree is labelled with pipeline, then pipelining can be used.
- For join operations, unless stated explicitly, only nested-loop-family join can always use pipelining.
  - If edge in evaluation tree is *NOT* labelled with pipelining, then it is assumed that pipelining cannot be used.
  - Annotations will be made explicit.

# Cost-based Query Optimisation

- Cost difference between evaluation plans for given query can be enormous
  - E.g. seconds vs. days in some cases
- Cost-based query optimisation
  - Find *logically equivalent expressions* of given expression
  - Estimate cost of all plans, e.g.
    - Choose algorithms to use for executing operator
    - Choose specific indices to use
  - Select detailed strategy with "least" cost (optimised plan)

Xi'an Jiaotong-Liverpool University 西交利物浦大学

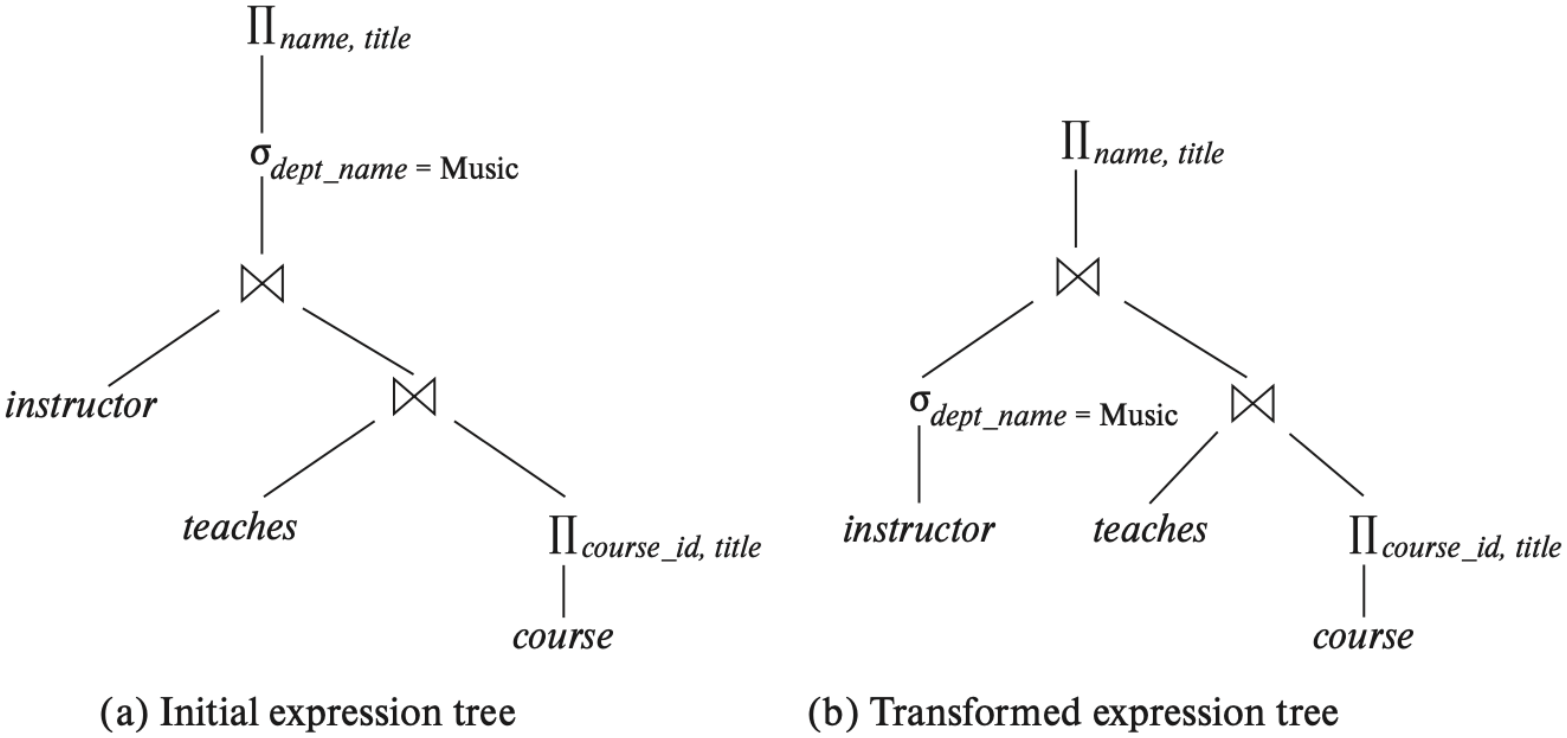
# Cost-based Query Optimisation – cont'd

- Estimation of plan cost based on:
  - Statistical information about relations, e.g. number of tuples, number of distinct values for attribute
  - Statistical estimation for intermediate results to compute cost of complex expressions
  - Cost formulae for algorithms, computed using statistics
- Since cost is estimate, selected plan is *not necessarily* least-costly plan;
  - However, as long as estimates are good, chosen plan will not be much more costly than least-costly one.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Equivalent expressions

- Alternative ways of evaluating given query
  - Equivalent expressions
  - Different algorithms for each operation




(a) Initial expression tree

(b) Transformed expression tree

Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Transformation of Relational Expressions

- Two relational algebra expressions are said to be equivalent if they generate same set of tuples on every legal database instance
  - Note: order of tuples is irrelevant
  - In SQL, inputs and outputs are multisets of tuples
  - Two expressions in multiset version of relational algebra are said to be equivalent if the two expressions generate same multiset of tuples on every legal database instance.
- Equivalence rule says that expressions of two forms are equivalent if
  - Can replace expression of first form by second, or vice versa

Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Equivalence Rules

- Rule 1: Conjunctive selection operations can be deconstructed into sequence of individual selections.

$$\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

- Rule 2: Selection operations are commutative.

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

- Rule 3: Only last one in sequence of projection operations is needed, others can be omitted.

$$\Pi_{L_1}(\Pi_{L_2}(\ldots(\Pi_{Ln}(E))\ldots)) = \Pi_{L_1}(E)$$

- Rule 4: Selections can be combined with Cartesian products and theta joins.
  - (a). $\sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$
  - (b). $\sigma_{\theta 1}(E_1 \bowtie_{\theta 2} E_2) = E_1 \bowtie_{\theta 1 \wedge \theta 2} E_2$

Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Equivalence Rules cont'd

- Rule 5: Theta-join operations (and natural joins) are commutative.

$$E_1 \bowtie_\theta E_2 = E_2 \bowtie_\theta E_1$$

- Rule 6.
  - (a) Natural join operations are associative:

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

  - (b) Theta joins are associative in following manner:

$$(E_1 \bowtie_{\theta 1} E_2) \bowtie_{\theta 2 \wedge \theta 3} E_3 = E_1 \bowtie_{\theta 1 \wedge \theta 3} (E_2 \bowtie_{\theta 2} E_3)$$

where $\theta_2$ involves attributes from only $E_2$ and $E_3$

Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Equivalence Rules cont'd

- Rule 7. Selection operation distributes over theta join operation under following two conditions:
  - (a) When $\theta_0$ involves only attributes of one of expressions ($E_1$) being joined.

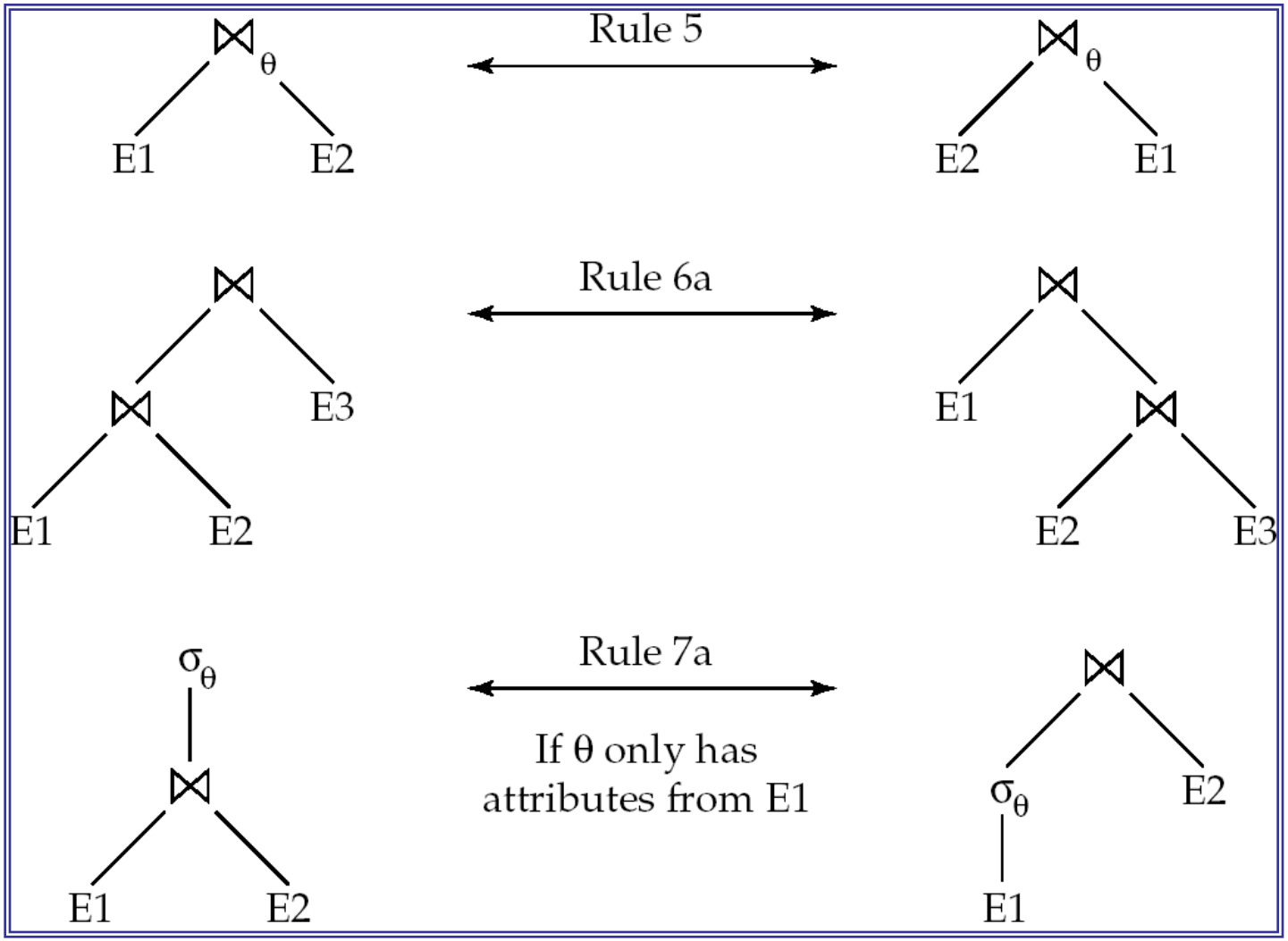
$$\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$$

  - (b) When $\theta_1$ involves only attributes of $E_1$ and $\theta_2$ involves only attributes of $E_2$.

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$$

Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Pictorial Depiction of Equivalence Rules




Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Equivalence Rules cont'd

- Rule 8. Projection operation distributes over theta join operation as follows:
  - (a) Let $L1$ and $L2$ be attributes from $E1$ and $E2$, if $\theta$ involves only attributes from $L_1 \cup L_2$:

$$\prod_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) = (\prod_{L_1}(E_1)) \bowtie_\theta (\prod_{L_2}(E_2))$$

  - (b) Consider join $E_1 \bowtie_\theta E_2$.
    - let $L_1$ and $L_2$ be sets of attributes from $E_1$ and $E_2$, respectively.
    - let $L_3$ be attributes of $E_1$ that are involved in join condition $\theta$, but are not in $L_1 \cup L_2$, and
    - let $L_4$ be attributes of $E_2$ that are involved in join condition $\theta$, but are not in $L_1 \cup L_2$.

$$\prod_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) = \prod_{L_1 \cup L_2}((\prod_{L_1 \cup L_3}(E_1)) \bowtie_\theta (\prod_{L_2 \cup L_4}(E_2)))$$

Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Equivalence Rules cont'd

- Rule 9. Set union and intersection are commutative (set difference is not commutative)

$$E_1 \cup E_2 = E_2 \cup E_1$$
$$E_1 \cap E_2 = E_2 \cap E_1$$

- Rule 10. Set union and intersection are associative (set difference is not associative).

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$
$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

- Rule 11. Selection operation distributes over $\cup$, $\cap$ and $-$.

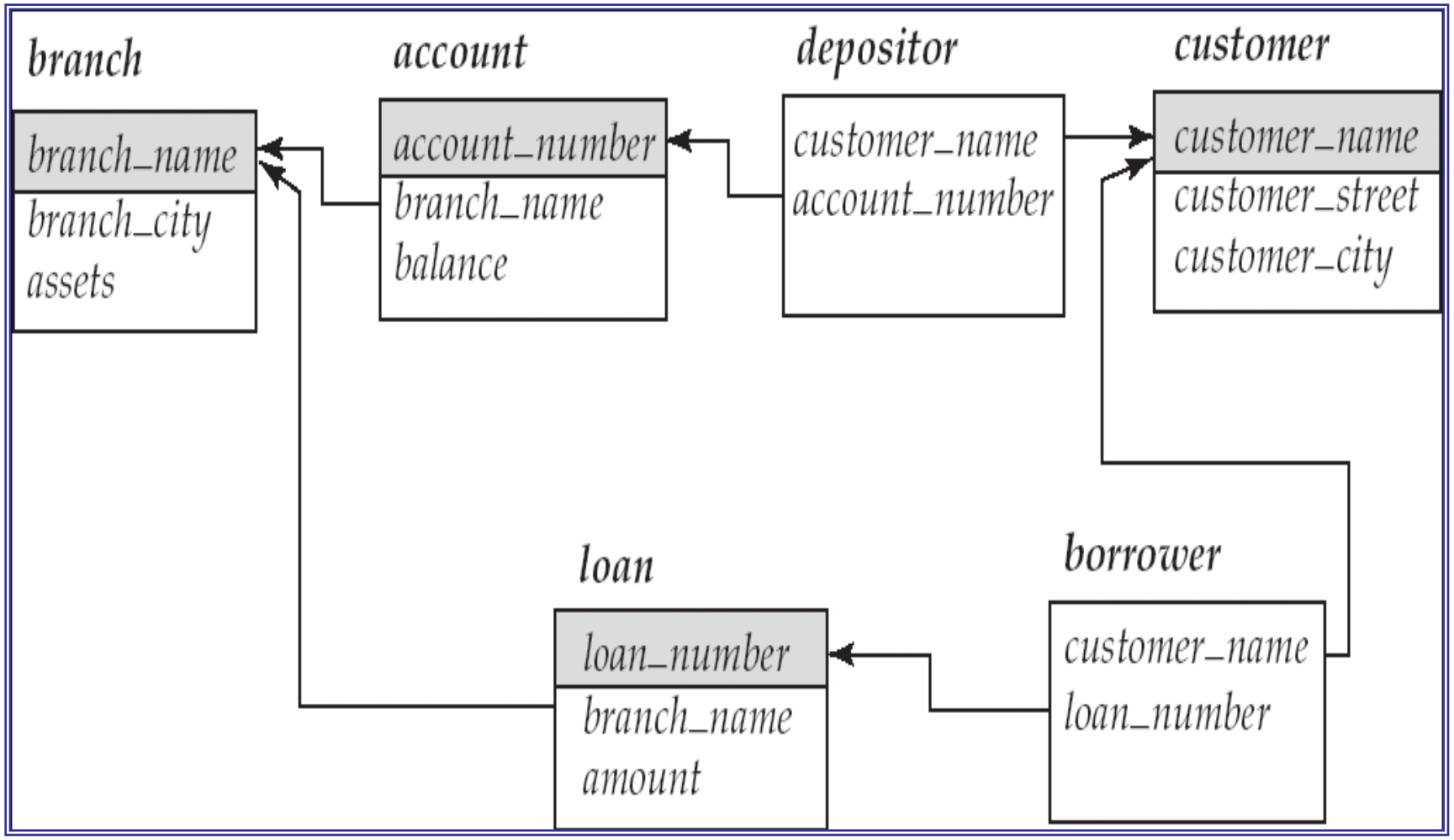
$$\sigma_\theta (E_1 \cup E_2) = \sigma_\theta (E_1) \cup \sigma_\theta (E_2)$$
$$\sigma_\theta (E_1 \cap E_2) = \sigma_\theta (E_1) \cap \sigma_\theta (E_2)$$
$$\sigma_\theta (E_1 - E_2) = \sigma_\theta (E_1) - \sigma_\theta (E_2)$$

- Rule 12. Projection operation distributes over union

$$\Pi_L(E_1 \cup E_2) = (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$

Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Banking Example




Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Example: Pushing Selections

- Query: *Find names of all customers who have account at some branches located in Brooklyn.*

$$\Pi_{customer_name}(\sigma_{branch_city = \text{"Brooklyn"}}(branch \bowtie (account \bowtie depositor)))$$

- Transformation using rule 7a (distribute selection).

$$\Pi_{customer_name}((\sigma_{branch_city = \text{"Brooklyn"}}(branch)) \bowtie (account \bowtie depositor))$$

- *Performing selection as early as possible reduces sizes of relations to be joined.*

Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Example: Multiple Transformations

- Query: *Find names of all customers with account at Brooklyn branch whose account balance is over $1000.*

$$\Pi_{customer_name}((\sigma_{branch_city = \text{"Brooklyn"} \wedge balance > 1000}(branch \bowtie (account \bowtie depositor)))$$

- Transformation using join associatively (Rules 6a and 7a):

$$\Pi_{customer_name}((\sigma_{branch_city = \text{"Brooklyn"} \wedge balance > 1000}(branch \bowtie account)) \bowtie depositor)$$

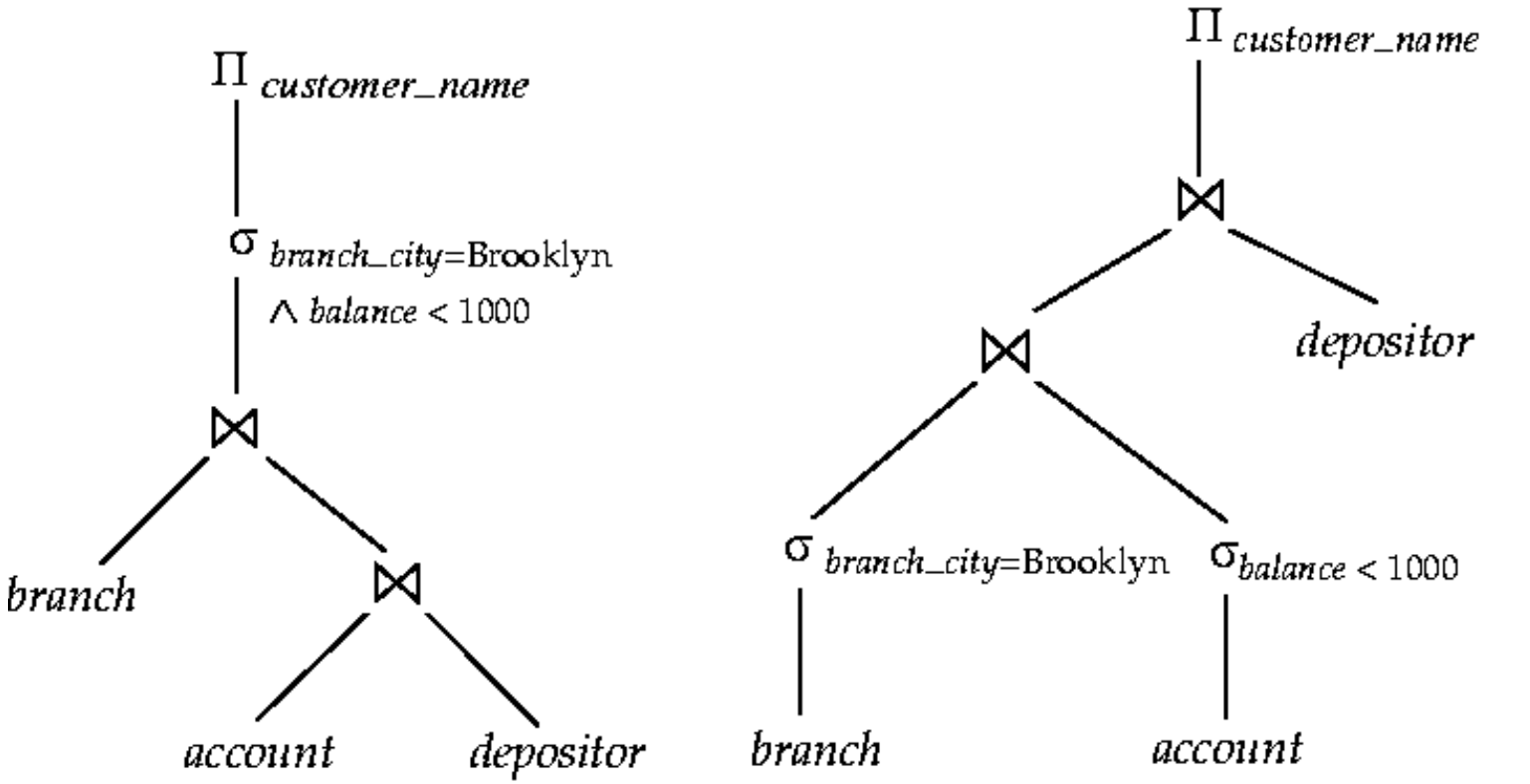
- Then, apply "perform selections early" (Rule 7b)

$$\sigma_{branch_city = \text{"Brooklyn"}}(branch) \bowtie \sigma_{balance > 1000}(account)$$

- Thus, sequence of transformations can be useful.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Multiple Transformations cont'd




(a) Initial expression tree

(b) Tree after multiple transformations

Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Transformation Example: Pushing Projections

$$\Pi_{customer_name}((\sigma_{branch_city = \text{"Brooklyn"}}(branch) \bowtie account) \bowtie depositor)$$

- When compute

$$(\sigma_{branch_city = \text{"Brooklyn"}}(branch) \bowtie account)$$

we obtain relation whose schema is:
(*branch_name, branch_city, assets, account_number, balance*)

- Push projections using equivalence rules 8b; eliminate unneeded attributes from intermediate results to get:

$$\Pi_{customer_name}((\Pi_{account_number}((\sigma_{branch_city = \text{"Brooklyn"}}(branch) \bowtie account)) \bowtie depositor))$$

*(HINT: L1 is null, L2 is customer_name; L3=L4=account_number)*

- *Performing projection as early as possible reduces sizes of tuples to be joined.*

Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Join Ordering Example

- For relations $r_1$, $r_2$, and $r_3$,

$$(r_1 \bowtie r_2) \bowtie r_3 = r_1 \bowtie (r_2 \bowtie r_3)$$

(Join Associativity)

- If $r_2 \bowtie r_3$ is quite large and $r_1 \bowtie r_2$ is small, may choose

$$(r_1 \bowtie r_2) \bowtie r_3$$

to compute and store smaller temporary relation.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Join Ordering Example cont'd

- Consider expression

  $\Pi_{customer_name}((\sigma_{branch_city = \text{"Brooklyn"}}(branch)) \bowtie (account \bowtie depositor))$

- Could compute "*account* $\bowtie$ *depositor*" first, and join result with

  $\sigma_{branch_city = \text{"Brooklyn"}}(branch)$

  but "*account* $\bowtie$ *depositor*" is likely to be large relation.

- Only small fraction of customers are likely to have accounts in branches located in Brooklyn, *better* to compute first

  $\sigma_{branch_city = \text{"Brooklyn"}}(branch) \bowtie account$

Xi'an Jiaotong-Liverpool University 西交利物浦大学

# Enumeration of Equivalent Expressions

- Query optimisers use equivalence rules to systematically generate expressions equivalent to given expression
- Very expensive in space and time

**procedure** genAllEquivalent($E$)
$EQ = \{E\}$
**repeat**
    Match each expression $E_i$ in $EQ$ with each equivalence rule $R_j$
    **if** any subexpression $e_i$ of $E_i$ matches one side of $R_j$
        Create a new expression $E'$ which is identical to $E_i$, except that
            $e_i$ is transformed to match the other side of $R_j$
        Add $E'$ to $EQ$ if it is not already present in $EQ$
**until** no new expression can be added to $EQ$

Xi'an Jiaotong-Liverpool University 西交利物浦大学

# End of Lecture

- Reading
  - Textbook chapter 16.1, 16.2, 16.3, 16.4